

CAHIER DES CHARGES • STAGE DE FIN D'ÉTUDES

# AutoCare AI

Application mobile intelligente de gestion  
et de maintenance automobile

React Native | Spring Boot 3 |  
PostgreSQL | Google Gemini API

Version 2.0 • **Juillet 2026**

Préparé par : Maghouan

Filière : Informatique / Génie Logiciel

# Sommaire

---

|           |                                         |          |
|-----------|-----------------------------------------|----------|
| <b>1</b>  | <b>Contexte</b>                         | <b>2</b> |
| <b>2</b>  | <b>Présentation du projet</b>           | <b>2</b> |
| <b>3</b>  | <b>Objectifs</b>                        | <b>2</b> |
| <b>4</b>  | <b>Problématique</b>                    | <b>3</b> |
| <b>5</b>  | <b>Solution proposée</b>                | <b>3</b> |
| <b>6</b>  | <b>Utilisateurs &amp; rôles</b>         | <b>3</b> |
| <b>7</b>  | <b>Fonctionnalités détaillées</b>       | <b>4</b> |
| 7.1       | Authentification & profil               | 4        |
| 7.2       | Gestion des véhicules                   | 4        |
| 7.3       | Gestion des composants                  | 4        |
| 7.4       | Historique de maintenance               | 4        |
| 7.5       | Scanner de facture intelligent (IA)     | 4        |
| 7.6       | Assistant IA conversationnel (Gemini)   | 4        |
| 7.7       | Maintenance prédictive                  | 5        |
| 7.8       | Notifications intelligentes             | 5        |
| 7.9       | Tableau de bord                         | 5        |
| 7.10      | Analyse financière                      | 5        |
| 7.11      | Estimation de la valeur du véhicule     | 5        |
| <b>8</b>  | <b>Exigences non fonctionnelles</b>     | <b>5</b> |
| <b>9</b>  | <b>Architecture technique</b>           | <b>5</b> |
| 9.1       | Stack technique                         | 6        |
| 9.2       | Pourquoi Gemini ?                       | 6        |
| 9.3       | Schéma d'architecture                   | 6        |
| 9.4       | Organisation du code                    | 7        |
| <b>10</b> | <b>Modèle de données</b>                | <b>7</b> |
| <b>11</b> | <b>Périmètre MVP vs Évolutions</b>      | <b>8</b> |
| <b>12</b> | <b>Livrables</b>                        | <b>8</b> |
| <b>13</b> | <b>Planning prévisionnel</b>            | <b>9</b> |
| <b>14</b> | <b>Risques et mesures d'atténuation</b> | <b>9</b> |
| <b>15</b> | <b>Critères d'acceptation</b>           | <b>9</b> |
| <b>16</b> | <b>Perspectives d'évolution</b>         | <b>9</b> |

## 1 Contexte

L'entretien régulier d'un véhicule est indispensable pour garantir sa sécurité, sa fiabilité et prolonger sa durée de vie. Cependant, la majorité des propriétaires ne disposent pas d'un historique structuré des opérations de maintenance effectuées sur leur véhicule.

Les informations sont souvent dispersées entre plusieurs factures, carnets papier ou souvenirs approximatifs, ce qui entraîne des oublis de maintenance, des dépenses imprévues et une diminution de la valeur de revente du véhicule.

L'intelligence artificielle générative offre aujourd'hui la possibilité d'automatiser une grande partie du suivi de maintenance, d'analyser l'état général du véhicule et d'assister le propriétaire dans ses décisions, via une simple conversation en langage naturel ou la lecture automatique de documents.

Ce projet consiste donc à développer une application mobile intelligente permettant de centraliser toutes les informations relatives à un véhicule tout en proposant des recommandations personnalisées grâce à l'IA (Google Gemini).

## 2 Présentation du projet

**AutoCare AI** est une application mobile permettant aux utilisateurs de gérer l'ensemble de leurs véhicules, de suivre leur historique de maintenance, de recevoir des rappels intelligents et de bénéficier d'une assistance conversationnelle basée sur l'intelligence artificielle (Gemini).

L'application constitue un véritable *carnet de santé numérique* pour chaque véhicule : chaque pièce mécanique est suivie individuellement, avec son état, son usure estimée et son historique d'intervention.

**Valeur ajoutée principale :** transformer un suivi papier/mémoire, dispersé et sujet à l'oubli, en un tableau de bord centralisé, prédictif et conversationnel.

## 3 Objectifs

L'application devra permettre de :

- Gérer plusieurs véhicules par utilisateur.
- Centraliser l'historique complet des interventions.
- Suivre l'état et le taux d'usure de chaque composant du véhicule.
- Générer automatiquement le profil complet d'un véhicule neuf.
- Simplifier l'ajout et la reconstitution de l'historique d'un véhicule d'occasion.
- Extraire automatiquement les données d'une facture scannée grâce à l'IA (OCR + Gemini).
- Prédire les prochaines opérations de maintenance à partir du kilométrage et de l'historique.

- Envoyer des notifications intelligentes et contextualisées.
- Produire des statistiques et une analyse des dépenses automobiles.
- Estimer la valeur actuelle de revente du véhicule.

## 4 Problématique

Les propriétaires de véhicules rencontrent plusieurs difficultés récurrentes :

| Difficulté rencontrée                    | Conséquence                                    |
|------------------------------------------|------------------------------------------------|
| Oubli des dates de maintenance           | Usure prématurée, panne évitable               |
| Perte des factures papier                | Impossible de prouver l'entretien à la revente |
| Méconnaissance de l'état réel des pièces | Décision de réparation mal informée            |
| Absence d'anticipation des pannes        | Coûts de réparation plus élevés                |
| Mauvais suivi budgétaire                 | Dépenses imprévues, absence de visibilité      |
| Manque d'historique lors de la revente   | Perte de valeur du véhicule                    |

## 5 Solution proposée

Développer une application mobile utilisant l'intelligence artificielle (Google Gemini) afin de :

- Créer automatiquement un carnet d'entretien numérique dès l'ajout du véhicule.
- Assister l'utilisateur en langage naturel dans la gestion de son véhicule.
- Automatiser le suivi des pièces mécaniques et le calcul de leur usure.
- Fournir des recommandations personnalisées selon l'usage réel du véhicule.
- Anticiper les futures opérations de maintenance et les risques de panne.

## 6 Utilisateurs & rôles

### Conducteur particulier

- Gestion de ses véhicules
- Consultation de l'historique
- Réception des rappels et notifications
- Analyse de ses dépenses
- Discussion avec l'assistant IA

### Administrateur

- Gestion des comptes utilisateurs
- Gestion des marques et modèles
- Gestion du référentiel de composants
- Consultation des statistiques globales
- Supervision de la plateforme

## 7 Fonctionnalités détaillées

### I Authentification & profil

- Création de compte (email / mot de passe)
- Connexion sécurisée (JWT + Refresh Token)
- Réinitialisation du mot de passe
- Gestion du profil utilisateur

### I Gestion des véhicules

**Ajout d'un véhicule** — informations collectées : marque, modèle, année, motorisation, type de carburant, transmission, kilométrage actuel, immatriculation, date d'achat.

#### Cas d'un véhicule neuf

Après validation, l'application crée automatiquement l'ensemble des composants standards du véhicule avec : état « Neuf », santé 100 %, date et kilométrage d'installation, durée de vie estimée. Composants créés notamment : huile moteur, filtre à huile, filtre à air, filtre habitacle, filtre carburant, batterie, pneus, plaquettes de frein, disques de frein, bougies, liquide de refroidissement, courroie de distribution, amortisseurs, embrayage, essuie-glaces, climatisation, éclairage, etc.

#### Cas d'un véhicule d'occasion

Les mêmes composants sont créés, mais avec un état initial « Inconnu ». L'utilisateur peut ensuite modifier manuellement les informations, scanner ses anciennes factures ou ajouter les interventions passées progressivement afin de reconstituer l'historique réel.

### I Gestion des composants

Chaque composant possède : nom, date d'installation, kilométrage d'installation, état, pourcentage d'usure, garantie, durée de vie estimée, historique des interventions liées.

### I Historique de maintenance

Pour chaque intervention : date, kilométrage, garage, description, pièces remplacées, coût, facture associée, garantie.

### I Scanner de facture intelligent (IA)

L'utilisateur photographie une facture. Le pipeline IA (OCR + Gemini) extrait automatiquement : date, garage, montant, kilométrage, pièces remplacées, main-d'œuvre. Les données structurées sont ensuite proposées à l'utilisateur pour validation avant enregistrement définitif.

### I Assistant IA conversationnel (Gemini)

L'utilisateur peut échanger en langage naturel avec l'application, par exemple : « J'ai changé les plaquettes de frein la semaine dernière ». L'assistant, propulsé par l'API Gemini, pose les questions complémentaires nécessaires (kilométrage, garage, coût) puis enregistre automatiquement l'intervention dans l'historique du composant concerné.

## I Maintenance prédictive

Le moteur IA analyse l'âge du véhicule, le kilométrage, les pièces déjà remplacées, les recommandations constructeur et l'historique complet, afin de calculer : les prochaines échéances de maintenance, les pièces à remplacer prochainement, et une estimation du risque de panne par composant.

## I Notifications intelligentes

Exemples : « Vidange dans 850 km », « Batterie âgée de 4 ans », « Assurance bientôt expirée », « Contrôle technique à prévoir ».

## I Tableau de bord

Affichage de : nombre de véhicules, dépenses mensuelles et annuelles, répartition des coûts par catégorie, coût par kilomètre, état de santé global du véhicule.

## I Analyse financière

Suivi des dépenses par catégorie : carburant, entretien, assurance, taxes, parking, lavage, réparations.

## I Estimation de la valeur du véhicule

L'IA estime la valeur actuelle de revente du véhicule selon son âge, son kilométrage, la qualité de son historique d'entretien et son état général.

## 8 Exigences non fonctionnelles

| Catégorie                   | Exigence                                                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------|
| Ergonomie                   | Interface moderne, intuitive, adaptée à un usage mobile rapide                                        |
| Performance                 | Temps de réponse API < 300 ms hors appels IA ; retour visuel systématique pendant les traitements IA  |
| Sécurité                    | Authentification JWT, chiffrement des mots de passe (BCrypt), validation des entrées, protection CORS |
| Confidentialité des données | Isolation stricte des données par utilisateur, minimisation des données envoyées à l'API Gemini       |
| Fiabilité                   | Sauvegarde automatique, gestion des erreurs et messages explicites                                    |
| Évolutivité                 | Architecture en couches, séparation claire des responsabilités (controller / service / repository)    |
| Compatibilité               | Application testée sur Android (et iOS si le temps le permet)                                         |
| Disponibilité               | Hébergement cloud avec objectif de disponibilité > 99 %                                               |

## 9 Architecture technique

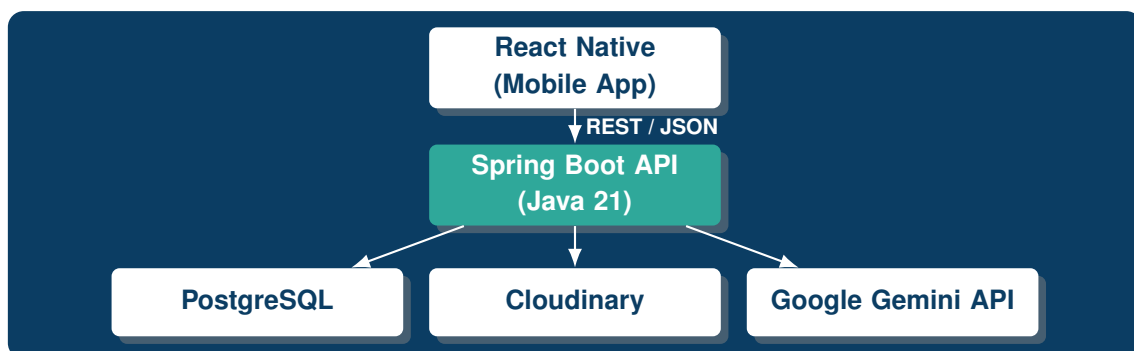
## Stack technique

| Couche                    | Technologies                                                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Frontend (mobile)         | React Native, TypeScript, React Navigation, TanStack Query (React Query), Axios, NativeWind (Tailwind CSS)              |
| Backend                   | Java 21, Spring Boot 3, Spring Security (JWT), Spring Data JPA / Hibernate, Spring Validation, Maven                    |
| Base de données           | PostgreSQL                                                                                                              |
| Authentification          | JWT Access Token + Refresh Token                                                                                        |
| Stockage fichiers         | Cloudinary (images des factures et véhicules)                                                                           |
| Intelligence artificielle | Google Gemini API (assistant conversationnel, extraction et structuration de données, aide à la maintenance prédictive) |
| OCR                       | Google ML Kit ou Tesseract OCR (pré-traitement des factures avant envoi à Gemini)                                       |
| Outils                    | Docker, Postman, Git & GitHub, Swagger/OpenAPI, Figma, IntelliJ IDEA, VS Code                                           |

## Pourquoi Gemini ?

L'API Google Gemini est utilisée pour deux usages complémentaires : (1) la **compréhension et structuration** du texte brut extrait d'une facture (dates, montants, pièces, garage) en JSON exploitable par le backend, et (2) l'**assistant conversationnel** qui interprète les messages libres de l'utilisateur et les transforme en actions concrètes (création d'une intervention, mise à jour d'un composant). Spring Boot appelle l'API Gemini via un client HTTP dédié (WebClient), sans dépendance à un SDK tiers non maintenu.

## Schéma d'architecture



## Organisation du code

|                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mobile — React Native</b><br>src/<br> -- components/<br> -- screens/<br> -- navigation/<br><br> -- services/<br> -- hooks/<br> -- context/<br> -- store/<br> -- utils/<br><br> -- assets/<br> -- types/<br>'-- constants/ | <b>Backend — Spring Boot</b><br>src/main/java<br> -- config/<br> -- controller/<br> -- service/<br><br> -- repository/<br> -- entity/<br> -- dto/<br> -- mapper/<br> -- security/<br><br> -- exception/<br> -- ai/ (client Gemini)<br> -- notification/<br>'-- utils/ |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 10 Modèle de données

Principales entités identifiées :

| Entité                | Rôle                                                         |
|-----------------------|--------------------------------------------------------------|
| Utilisateur           | Compte, rôle (conducteur / admin), informations de connexion |
| Véhicule              | Informations générales du véhicule, lié à un utilisateur     |
| Marque / Modèle       | Référentiel constructeur                                     |
| Composant             | Pièce suivie individuellement (état, usure, durée de vie)    |
| HistoriqueMaintenance | Intervention réalisée sur un composant                       |
| Facture               | Document scanné, données extraites par l'IA                  |
| Dépense               | Ligne financière rattachée à une catégorie                   |
| Notification          | Alerte générée pour l'utilisateur                            |
| Garantie              | Couverture associée à un composant ou une intervention       |
| ConversationIA        | Historique des échanges avec l'assistant Gemini              |

## 11 Périmètre MVP vs Évolutions



| Fonctionnalité                                        | Statut              |
|-------------------------------------------------------|---------------------|
| Authentification JWT                                  | MVP                 |
| Gestion de plusieurs véhicules                        | MVP                 |
| Création automatique des composants (neuf / occasion) | MVP                 |
| Historique des interventions                          | MVP                 |
| Scanner de factures (OCR + Gemini)                    | MVP                 |
| Assistant IA conversationnel simple                   | MVP                 |
| Tableau de bord & dépenses                            | MVP                 |
| Notifications intelligentes                           | MVP(règles simples) |
| Maintenance prédictive avancée                        | V2                  |
| Estimation de la valeur de revente                    | V2                  |
| Notifications push natives                            | V2                  |

Ce découpage MVP / V2 permet de sécuriser une soutenance sur un périmètre réaliste tout en gardant une vision produit ambitieuse.

## 12 Livrables

- Application mobile Android (build de démonstration)
- API REST documentée (Swagger/OpenAPI)
- Base de données PostgreSQL avec scripts de migration
- Documentation technique (architecture, choix techniques)
- Documentation utilisateur
- Rapport de stage
- Support de présentation / soutenance

## 13 Planning prévisionnel

| Période   | Objectifs                                                                          |
|-----------|------------------------------------------------------------------------------------|
| Semaine 1 | Analyse des besoins, cahier des charges, maquettage UI/UX (Figma)                  |
| Semaine 2 | Modélisation base de données, mise en place Spring Boot, authentification JWT      |
| Semaine 3 | Gestion des véhicules, gestion des composants, historique de maintenance           |
| Semaine 4 | Intégration OCR, intégration API Gemini (scanner + assistant), notifications       |
| Semaine 5 | Tableau de bord, tests, corrections, déploiement, finalisation de la documentation |

## 14 Risques et mesures d'atténuation

| Risque                                             | Impact | Mesure d'atténuation                                                             |
|----------------------------------------------------|--------|----------------------------------------------------------------------------------|
| Quota / coût de l'API Gemini                       | Moyen  | Mise en cache des réponses, limitation du volume de tokens, mode dégradé sans IA |
| Qualité variable de l'OCR sur factures manuscrites | Moyen  | Validation manuelle obligatoire avant enregistrement                             |
| Périmètre trop large pour la durée du stage        | Élevé  | Priorisation stricte MVP / V2 (voir section 11)                                  |
| Sécurité des données personnelles                  | Élevé  | JWT, chiffrement, isolation des données par utilisateur                          |

## 15 Critères d'acceptation

- Un utilisateur peut créer un compte, se connecter et ajouter un véhicule neuf avec génération automatique de ses composants.
- Un utilisateur peut scanner une facture et voir les champs extraits automatiquement avant validation.
- L'assistant IA peut enregistrer une intervention simple décrite en langage naturel.
- Le tableau de bord affiche correctement les dépenses totales et leur répartition par catégorie.
- Une notification est générée lorsque le kilométrage restant avant une vidange passe sous un seuil défini.

## 16 Perspectives d'évolution

- Connexion Bluetooth OBD-II pour lecture directe des données véhicule.
- Détection de pannes à partir des codes défauts.

- Détection de l'usure des pneus via analyse d'image.
- Portail dédié aux garages partenaires.
- Marketplace de pièces automobiles.
- Synchronisation avec les compagnies d'assurance.
- Assistant vocal intelligent.
- QR Code contenant l'historique complet du véhicule pour faciliter la revente.